LEAP Hand 큐브 면 맞추기—실기 이관·평가 가이드

과제 2 (LeapCubeFaceUp) / 시뮬 학습 → 실기 평가

2026-08-30

Contents

0. 전제: 실기에서 RL 학습은 하지 않습니다 2

1. 배포 산출물 2

2. 제어 규약 3
- 2.1 특히 주의할 두 가지 . . . 3
- 2.2 제어 루프 의사코드 . . . 3

3. 관절 순서와 명령 범위 3

3b. 실기 모터 매핑—이 손은 순서가 세 겹으로 다릅니다 4
- 3b.1 어긋나는 지점 . . . 4
- 3b.2 확정 매핑 . . . 4
- 3b.3 부호 - 13 개는 확인됐고 3 개는 미확인입니다 . . . 5
- 3b.4 하필 그 3 개가 게인도 다릅니다 . . . 5
- 3b.5 set_ones() 를 쓰면 안 됩니다 . . . 5
- 3b.6 토크 - 실기 설정이 시뮬보다 높습니다 . . . 5

4. 관측 108 차원의 구성 5

5. 비전 - 마커와 카메라 6
- 5.1 정확도 요구가 두 갈래입니다 . . . 6
- 5.2 마커 - 3x3 루빅큐브는 중앙 스티커만 가능합니다 . . . 6
- 5.3 면별 마커 회전 보정 . . . 7
- 5.4 카메라 캘리브레이션 . . . 7

5b. 카메라 여러 대 연결하기 7
- 5b.1 배치 . . . 7
- 5b.2 장치 이름을 고정하십시오 . . . 7
- 5b.3 USB 대역폭 - 여기서 가장 많이 막힙니다 . . . 8
- 5b.4 노출을 수동으로 고정하십시오 . . . 8
- 5b.5 최신 프레임만 쓰십시오 . . . 8
- 5b.6 카메라 간 외부 파라미터가 필요 없습니다 . . . 9
- 5b.7 융합 . . . 9
- 5b.8 끊겼을 때 . . . 9
- 5b.9 시뮬에 반영할 것 . . . 9
- 5b.10 검증 . . . 9

5c. 배포 패키지 (Leap_Test) 9
- 5c.1 정책이 numpy 로 돕니다 . . . 10
- 5c.2 관측 조립기는 시뮬과 대조했습니다 . . . 10
- 5c.3 상수를 손으로 옮기지 않습니다 . . . 10

5c.4 반드시 재서 넣어야 하는 값 두 개 . . . 10
5c.4 실행 순서 . . . 11

6. 계단식 시험 **11**
1 단계—개루프 재생 (큐브 없이) . . . 11
2 단계—정지 파지 . . . 11
3 단계—단일 목표 1 회 . . . 11
4 단계—연쇄 . . . 12

7. 안전 **12**

8. 평가 프로토콜 **12**
8.1 표본 수—여기서 기대를 낮춰야 합니다 . . . 12
8.2 사전 등록 . . . 12
8.3 시행 절차 . . . 12

9. 시뮬 기준선 - 실기에서 무엇과 비교하는가 **12**
9.1 정책별 성능 (토크 0.11 N·m) . . . 13
9.2 토크 - 지배적인 변수이고, 증류는 손해입니다 . . . 13
9.3 sim2real 항목별 비용 . . . 13
9.4 작동 지연 비용 . . . 14

10. 이관 전에 메울 구멍 **14**
10.1 소프트웨어 / 설정 . . . 14
10.2 시뮬에 없는 것 . . . 14
10.3 실기에서 확인해야 확정되는 것 . . . 15

11. 체크리스트 **15**

0. 전제: 실기에서 RL 학습은 하지 않습니다

이 문서는"실기 학습 가이드"라는 이름을 달고 있지만, **실기에서 강화학습을 돌리는 절차서가 아닙니다.** 그것이 불가능하기 때문입니다.

우리 학습량은 시드당 4,000 만 스텝이고 제어 주기는 20 Hz 입니다.

$$4 \times 10^7 \text{ 스텝} \times 0.05 \text{ s} = 2 \times 10^6 \text{ s} \approx 23 \text{ 일}$$

리셋과 사람의 개입을 전부 빼고 **연속 구동 23 일, 시드 하나**입니다. 같은 학습이 시뮬에서는 1 시간 36 분에 끝납니다. 큐브를 떨어뜨릴 때마다 사람이 다시 올려줘야 한다는 점을 넣으면 실제 소요는 이보다 훨씬 큽니다.

따라서 실기의 역할은 셋으로 한정됩니다.

1. **평가** —시뮬에서 고른 정책이 실기에서 얼마나 남는지 측정한다.
2. **시스템 식별** —지연·마찰·토크 같은 미지 파라미터를 실측해 시뮬에 되먹인다.
3. **회귀 검증** —시뮬 변경이 실기를 망가뜨리지 않았는지 확인한다.

정책 **선택은 전부 시뮬에서 끝냅니다.** 이유는 8 절의 표본 수 계산에 있습니다.

1. 배포 산출물

정책 파일	`distill_t2_v57/policy`
형식	flax 직렬화 `{"norm": ..., "policy": ...}`
관측	108 차원 실수 벡터
행동	16 차원, 각 $[-1, 1]$
은닉층	512, 256, 128

추론	결정적 (deterministic), 분포의 평균만 사용

`norm` 을 반드시 함께 옮겨야 합니다. 이것은 관측 정규화 통계 (평균·표준편차) 로, 없으면 정책은 정규화되지 않은 입력을 받아 무의미한 출력을 냅니다. 신경망 파라미터만 챙기고 정규화 통계를 빠뜨리는 것이 이관에서 가장 흔한 실패입니다. 정책이"학습이 안 된 것처럼"움직이면 먼저 이것을 의심하십시오.

2. 제어 규약

실기 이관에서 깨지는 것은 대부분 정책이 아니라 규약입니다. 아래는 학습 코드가 전제하는 값이며, **하나라도 어긋나면 정책은 다른 플랜트를 제어하게 됩니다.**

항목	값	근거
제어 주기	**20 Hz**	`ctrl_dt = 0.05`
물리 적분	100 Hz (5 substep)	`sim_dt = 0.01`
행동 해석	$\Delta = a \times 0.2$ rad	`action_scale = 0.2`
누적 대상	**직전 명령각** (측정각 아님)	`data.ctrl + delta`
명령 필터	**없음**	`ema_alpha = 1.0`
명령 클리핑	3 절 표	`safe_lowers/uppers`
단위	라디안	
이력 길이	3 프레임	`history_len = 3`
초기 안정화	8 스텝 (0.4 초) 무동작	`settle_steps = 8`
토크 상한	0.11 N·m	`actuator_force_limit`

2.1 특히 주의할 두 가지

누적은 명령각에 합니다. 다음 목표각은"직전에 **보낸** 목표각 + Δ"이지"지금 **측정된** 관절각 + Δ"가 아닙니다. 후자로 구현하면 서보 추종 오차가 매 스텝 누적 경로에 섞여 들어가 전혀 다른 동역학이 됩니다. 학습은 전자를 전제했습니다.

추가 필터를 넣지 마십시오. `ema_alpha = 1.0` 이라 학습 시 명령 평활이 꺼져 있었습니다. 실기에서 손이 떨린다고 저역통과 필터를 얹으면 학습 조건에서 벗어납니다. 떨림은 정책의 성질이며 (6 절), 필터로 덮으면 재파지 동작까지 함께 느려집니다.

2.2 제어 루프 의사코드

```
q_cmd = 초기자세                          # 16차원, rad
hist  = 0으로 채운 이력 버퍼
for 8스텝: 물리만 진행 (정책 호출 없음)   # settle

매 50 ms:
    q_meas = 엔코더 읽기                  # 16차원, rad
    obs    = 관측조립(q_meas, q_cmd, hist, 큐브자세, 직전행동)
    a      = 정책(정규화(obs, norm))      # 16차원, [-1,1]
    q_cmd  = clip(q_cmd + 0.2 * a, 하한, 상한)
    서보에 q_cmd 전송
    이력갱신(hist, q_meas - q_cmd_직전)
```

3. 관절 순서와 명령 범위

순서가 어긋나면 **오류 없이 조용히 망가집니다.** 아래 순서를 그대로 따르십시오.

인덱스 0-15: `if_mcp, if_rot, if_pip, if_dip, mf_mcp, mf_rot, mf_pip, mf_dip, rf_mcp, rf_rot, rf_pip, rf_dip, th_cmc, th_axl, th_mcp, th_ipl`

(if = 검지, mf = 중지, rf = 약지, th = 엄지)

관절 부류	하한 (rad)	상한 (rad)	비고
mcp	-0.314	2.230	
rot	-1.047	1.047	
pip	**-0.200**	1.885	원 범위 -0.506 에서 상향
dip	**-0.200**	2.042	원 범위 -0.366 에서 상향
th_cmc	-0.349	2.094	
th_axl	-0.470	2.443	공식 URDF 기준으로 수정됨
th_mcp	-1.200	1.900	공식 URDF 기준으로 수정됨
th_ipl	-1.340	1.880	

pip·dip 하한 -0.200 은 의도적으로 좁힌 값입니다. 원래 범위를 그대로 두면 정책이 끝마디를 -21° 까지 젖혀 사람이 만들 수 없는 자세를 시간의 44.9% 동안 사용했습니다. 범위로 막자 0.00% 가 되었고 성능 손실은 0.4%p 였습니다. **실기 클리핑도 반드시 이 값을 써야 합니다.** 원 범위를 쓰면 과신전이 실기에서 되살아나고, 감속기에 무리가 갑니다. 엄지는 사람도 IP 과신전이 가능하므로 제외했습니다.

3b. 실기 모터 매핑—이 손은 순서가 세 겹으로 다릅니다

`leap_python/main.py` 의 배선 주석과 `leap_hand_utils.py` 의 관절 범위표를 대조해 확정한 매핑입니다.

3b.1 어긋나는 지점

1. **이 손은 모터 ID 를 1~16 으로, 공식 순서를 완전히 뒤집어 부여했습니다.** 공식은 0~15, 검지-> 중지-> 약지-> 엄지, 각 손가락 MCP Side->Forward->PIP->DIP. 이 손은 엄지-> 약지-> 중지-> 검지, 각 손가락 DIP->PIP->Forward->Side.
2. **시뮬은 손가락 안에서 앞 두 개가 공식과 반대입니다.** 우리 MJCF 는 손바닥에서 바깥 순서 (굴곡 먼저) 이고 공식은 Side 를 먼저 셉니다. 엄지는 두 순서가 일치합니다.
3. **영점이 pi 만큼 다릅니다.** LEAP 은 3.14159 rad 가 편 자세입니다.

3b.2 확정 매핑

sim	관절	역할	공식 ID	모터 ID	배열 위치
0	if_mcp	검지 MCP Fwd	1	15	14
1	if_rot	검지 MCP Side	0	16	15
2	if_pip	검지 PIP	2	14	13
3	if_dip	검지 DIP	3	13	12
4	mf_mcp	중지 MCP Fwd	5	11	10
5	mf_rot	중지 MCP Side	4	12	11
6	mf_pip	중지 PIP	6	10	9
7	mf_dip	중지 DIP	7	9	8
8	rf_mcp	약지 MCP Fwd	9	7	6
9	rf_rot	약지 MCP Side	8	8	7
10	rf_pip	약지 PIP	10	6	5
11	rf_dip	약지 DIP	11	5	4
12	th_cmc	엄지 MCP Side	12	4	3
13	th_axl	엄지 MCP Fwd	13	3	2
14	th_mcp	엄지 PIP	14	2	1
15	th_ipl	엄지 DIP	15	1	0

```
SIM2ARR = [14, 15, 13, 12, 10, 11, 9, 8, 6, 7, 5, 4, 3, 2, 1, 0]
```

```
hw[SIM2ARR[i]] = q_sim[i] + 3.14159        # 명령
q_sim[i]       = hw[SIM2ARR[i]] - 3.14159  # 읽기
```

이 순열을 학습 코드가 아니라 드라이버 경계면에 두십시오. 시뮬의 `JOINT_NAMES` 순서를 바꾸면 신경망 1 층 가중치가 입력 순서에, 마지막 층이 출력 순서에 묶여 있으므로 **학습한 체크포인트가 전부 무효**가 됩니다.

3b.3 부호 - 13 개는 확인됐고 3 개는 미확인입니다

`leap_hand_utils.py` 가"모든 회전 방향 부호는 두 손 사이에 같다. 오프셋만 다르다"고 명시합니다. 그리고 범위표가 부호를 지문처럼 확인해 줍니다 - 부호가 뒤집히면 `[-0.314, 2.23]` 이 `[-2.23, 0.314]` 로 거울상이 되어야 하는데, 16 개 중 13 개가 **비대칭 범위**이고 그대로 일치했습니다.

`if_rot, mf_rot, rf_rot` 만 +-1.047 로 대칭이라 이 방법으로 확인되지 않습니다. 실기에서 직접 봐야 합니다. `leap_python/test_each_joint.py` 가 정확히 그 용도로 이미 작성돼 있습니다 - 배열 위치 15(모터 ID 16) 를 움직여 **검지가 굽으면** 이 표가 틀린 것이고, **옆으로 벌어지면** 맞습니다.

틀리면 검지·중지·약지의 굽힘 명령과 벌림 명령이 통째로 바뀌어, 쥐려 할 때 손가락이 벌어집니다.

3b.4 하필 그 3 개가 게인도 다릅니다

실기 `main.py` 는 옆흔들림 모터 (ID 8·12·16) 에만 `kP x 0.75, kD x 0.75` 를 적용합니다. 우리 MJCF 는 `<position kp="3.0">` 하나뿐이라 16 개가 모두 같습니다. **부호 미확인과 게인 불일치가 같은 3 개 관절에 겹칩니다.** 개루프 재생에서 가장 먼저 확인할 대상입니다.

3b.5 set_ones() 를 쓰면 안 됩니다

`lhu.sim_ones_to_LEAPhand` 는 [-1,1] 을 관절 범위로 스케일하는 **절대 위치** 변환입니다. 우리 정책의 출력은 **증분** (`q_cmd += 0.2 * a`) 입니다. 그대로 넣으면 매 스텝 손이 전 범위를 튀어다닙니다. `q_cmd` 를 직접 누적한 뒤 `set_leap()` 으로 절대 각도를 보내십시오.

3b.6 토크 - 실기 설정이 시뮬보다 높습니다

MJCF 주석에 환산식이 있습니다: `max torque = 0.2196 = 600/1000*0.366`. 즉 토크 [N·m] = 전류 [A] x 0.366.

전류	토크	
450 mA	**0.165 N·m**	실기 `main.py` 의 `curr_lim`
550 mA	0.201 N·m	full hand 권장
600 mA	0.220 N·m	MJCF 기준값
-	**0.110 N·m**	우리 시뮬 기본값

실기 동작점이 시뮬보다 50% 높습니다. 9 절의 토크 의존성 표를 보면 이 차이가 성능을 크게 바꿉니다. 다만 확인이 필요합니다 - 0.366 이 실제 장착 모터의 토크 상수인지, 전류 상한은 상시 출력이 아니라 상한이며 거기 붙어 돌면 과열한다는 점, 기어 효율 반영 여부입니다.

4. 관측 108 차원의 구성

항목	차원	실기 출처
관절각	16	엔코더
관절 오차 이력 (3 프레임)	48	엔코더 - 명령각
직전 행동	16	내부 값
큐브 위치 오차 이력 (3 프레임)	9	**외부 감지 필요**
큐브 자세 오차 이력 (3 프레임)	18	**외부 감지 필요**
자세 오차 스칼라	1	**외부 감지 필요**

74% 는 이미 고유수용성입니다. 외부 감지가 필요한 것은 28 차원 (26%) 뿐입니다.

관절 오차 이력은 측정각 − 명령각입니다. 부호와 기준을 혼동하지 마십시오.

5. 비전 - 마커와 카메라

5.1 정확도 요구가 두 갈래입니다

학습 시 관측에 주입한 잡음과, 성공 판정 임계값이 다릅니다.

	정책이 견디도록 학습된 잡음	성공 판정 임계값
관절각	+-0.05 rad (+-2.9 도)	-
큐브 위치	**+-0.02 m**	**0.015 m**
큐브 자세	약 0.1 (약 5.7 도)	**0.15 rad (8.6 도)**

정책 입력의 허용 잡음 (+-2cm) 이 성공 임계값 (1.5cm) 보다 큽니다. 결함이 아니라 의도된 비대칭입니다. 정책은 흐린 눈으로 움직이고, 판정은 깨끗한 상태로 합니다.

- **정책 입력용** - +-2 cm / +-5.7 도면 충분합니다. 온라인 20 Hz.
- **판정용** - 1.5 cm / 0.15 rad 보다 확실히 정확해야 합니다.

판정은 온라인일 필요가 없습니다. 시행을 녹화해 두고 사후에 다중 프레임 평균과 서브픽셀 정밀화를 걸면 온라인 경로보다 훨씬 정확합니다. 그리고 성공 조건이 20 스텝 (1 초) 유지이므로 판정 시점에는 큐브가 거의 정지해 있고 프레임이 30 장 쌓입니다. **추적은 어렵고 판정은 쉽습니다.**

5.2 마커 - 3x3 루빅큐브는 중앙 스티커만 가능합니다

루빅큐브에서 **코어에 고정된 것은 6 개 중앙 조각뿐**입니다. 코너·엣지는 면을 돌리면 움직이므로 마커를 붙이면 보드 정의가 깨집니다. 면을 가로지르는 마커는 한 층만 돌려도 찢어집니다.

마커는 스티커의 **75%** 까지입니다. 4x4 사전은 6x6 셀이라 콰이어트 존 1 셀을 확보하려면 `marker + 2*(marker/6) <= sticker`, 즉 `marker <= 0.75*sticker` 입니다.

스티커	최대 마커	720p/30cm	720p/40cm	1080p/30cm
15 mm	11.25 mm	42 px	31 px	62 px
16 mm	12.00 mm	44 px	33 px	67 px
17 mm	12.75 mm	47 px	35 px	71 px

4x4 사전의 검출 하한은 약 40 px 입니다. **720p 를 40cm 에 두면 미달입니다.** 30cm 까지 당기면 아슬아슬하게 되고, 1080p 면 여유가 있습니다.

ID = 면 번호입니다: 0=+z, 1=-z, 2=+x, 3=-x, 4=+y, 5=-y. 손 기준 마커는 ID 40, 30 mm 입니다.

만들기

```
leap_python/marker/
  make_center_templates.py   # 중앙 스티커용 (3x3 루빅큐브)
  center_markers_A4.png      # 스티커 15/16/17mm 3종 + 손 마커
  cube_board.py              # 보드 3D 좌표 정의
  calibrate_camera.py        # 카메라 내부 파라미터
  calibrate_face_rot.py      # 면별 마커 회전 추정
  track_cube.py              # 실행 시 자세 추정
```

1. `center_markers_A4.png` 를 **실제 크기 100%** 로 인쇄합니다. 무광 용지.
2. 시트 맨 위 **50 mm 눈금자를 자로 확인**합니다. 다르면 배율이 틀린 것이고, 그대로 쓰면 거리 추정이 그 비율만큼 통째로 틀립니다.

3. 큐브 중앙 스티커를 재고 맞는 줄만 씁니다.
4. 타일 바깥 사각선을 따라 자릅니다. 스티커와 같은 크기라 기존 스티커 위에 그대로 덮입니다.
5. 큐브에 축을 정하고 (한 면 +z, 인접 두 면 +x·+y 로 오른손 좌표계) 해당 ID 를 그 면 중앙에 붙입니다.
6. **방향은 아무렇게나 붙여도 됩니다.** 5.3 이 자동으로 잡습니다.

5.3 면별 마커 회전 보정

12 mm 급 마커에는 방향 표시를 넣을 여백이 없습니다 (콰이어트 존을 먹습니다). 그리고 **면을 돌리면 그 면의 중앙 조각이 제자리에서 90 도 회전**합니다. 위치와 법선은 그대로라 면 오차 (스핀 불변) 에는 영향이 없지만, 두 면이 동시에 보일 때 보드 적합이 어긋납니다.

`calibrate_face_rot.py` 가 후보 4^6 = 4096 개를 전수 탐색해 재투영 오차가 최소인 조합을 고릅니다. **큐브를 여러 자세로 들고 두 면 이상 보이는 프레임 15 장**이면 됩니다. 한 면만 보이는 프레임은 그 면의 회전이 자세와 구별되지 않아 정보가 없으므로 버립니다.

합성 검증에서 코너 잡음 1 px 까지 참값을 정확히 복원했습니다.

스크램블한 뒤에는 이것을 다시 돌리십시오.

5.4 카메라 캘리브레이션

체스보드를 9x6 내부코너로 A4 에 100% 인쇄하고 한 칸을 자로 재서 `SQUARE_MM` 에 넣습니다. 이 값이 틀리면 이후 모든 거리 추정이 같은 비율로 틀립니다.

```
python calibrate_camera.py "calib/*.jpg" camera0.npz
```

각도·위치를 바꿔가며 20~30 장. 화면 구석까지 채워야 왜곡 계수가 나옵니다. 재투영 오차가 **0.5 px 이하면 양호, 1.0 px 를 넘으면 다시** 찍으십시오. **카메라마다 따로** 해야 합니다.

5b. 카메라 여러 대 연결하기

중앙 스티커 배치는 **면당 마커가 1 개**이고, 하필 **손가락이 가장 자주 닿는 자리** 입니다. 가림이 지배적인 실패 원인이므로 카메라를 늘리는 것이 가장 효과가 큽니다.

5b.1 배치

두 대를 손 주위로 **약 90 도 벌려** 놓고 둘 다 약간 내려다보게 하십시오. 같은 쪽에 두면 가리는 면이 겹쳐 효과가 없습니다. 깊이 (광축 방향) 오차가 가로세로보다 3~5 배 크므로, 한 대가 못 재는 축을 다른 대가 가로로 재주는 배치가 이상적입니다.

5b.2 장치 이름을 고정하십시오

`/dev/video0`, `/dev/video1` 같은 번호는 **부팅·재연결 순서에 따라 바뀝니다.** 그대로 코드에 박으면 어느 날 두 카메라가 뒤바뀌고, 내부 파라미터가 서로 어긋난 채로 조용히 잘못된 자세가 나옵니다.

```
v4l2-ctl --list-devices        # 어떤 노드가 어떤 카메라인지 묶어서 보여준다
ls -l /dev/v4l/by-id/          # 안정적인 심볼릭 링크
```

최신 커널은 카메라 하나가 video 노드를 2 개 이상 만듭니다 (영상 + 메타데이터). **번호가 연속이라고 카메라 2 대가 아닙니다.** `--list-devices` 로 확인하십시오.

코드에서는 by-id 경로를 쓰십시오.

```
CAMS = ["/dev/v4l/by-id/usb-...-video-index0",
        "/dev/v4l/by-id/usb-...-video-index0"]
cap = cv2.VideoCapture(path, cv2.CAP_V4L2)
```

5b.3 USB 대역폭 - 여기서 가장 많이 막힙니다

UVC 카메라 두 대를 같은 USB 2.0 호스트에 물리고 720p 무압축 (YUYV) 으로 열면 대역폭 할당에 실패합니다.

```
VIDIOC_STREAMON: No space left on device
```

해결 순서

1. **MJPG 로 강제하십시오.** 압축 포맷이라 대역폭이 10 분의 1 수준입니다. 해상도 설정 **전에** FOURCC 를 걸어야 합니다.

```
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*"MJPG"))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
cap.set(cv2.CAP_PROP_FPS, 30)
```

2. **서로 다른 USB 루트 허브에 꽂으십시오.** `lsusb -t` 로 트리를 보고 같은 버스에 몰리지 않게 합니다. 노트북 내장 카메라는 보통 별도 버스라 유리합니다.
3. 그래도 안 되면 해상도나 프레임률을 낮춥니다.

실제로 무엇이 열렸는지 확인하십시오. 설정은 조용히 무시될 수 있습니다.

```
import struct
fourcc = int(cap.get(cv2.CAP_PROP_FOURCC))
print(struct.pack("<I", fourcc).decode(), cap.get(cv2.CAP_PROP_FRAME_WIDTH),
      cap.get(cv2.CAP_PROP_FPS))
```

5b.4 노출을 수동으로 고정하십시오

노트북 웹캠은 어두우면 자동으로 노출 시간을 늘려 30 fps 가 15 fps 로 떨어집니다. 그러면 프레임 간격 66 ms 로 **제어 주기 (50 ms) 보다 느려집니다.** 그리고 카메라 두 대의 노출이 각각 다르게 흔들려 색과 밝기가 어긋납니다.

```
cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 1)   # 1= 수동 (드라이버마다 값이 다름)
cap.set(cv2.CAP_PROP_EXPOSURE, -6)
cap.set(cv2.CAP_PROP_AUTOFOCUS, 0)
```

조명을 밝고 고르게 하고, 마커에 정반사가 생기지 않게 하십시오.

5b.5 최신 프레임만 쓰십시오

`read()` 를 제어 루프에서 그냥 부르면 드라이버 큐에 쌓인 **오래된 프레임**을 꺼내게 되어 지연이 누적됩니다.

카메라마다 스레드를 하나 두고 계속 읽으면서 최신 프레임만 보관하십시오.

```
class Grabber(threading.Thread):
    def __init__(self, cap):
        super().__init__(daemon=True)
        self.cap, self.lock, self.latest = cap, threading.Lock(), None
    def run(self):
        while True:
            ok, f = self.cap.read()
            if ok:
                with self.lock:
                    self.latest = (time.time(), f)
    def get(self):
        with self.lock:
            return self.latest
```

카메라 간 시차를 줄이려면 모든 카메라에 `grab()` 을 연달아 건 뒤 `retrieve()` 를 부르는 방법도 있습니다. 웹캠에는 하드웨어 동기가 없지만, 우리 손은 토크 제한으로 느리고 1 프레임 (33 ms) 시차는 이미 무작위화하는 지연 예산 안에 들어갑니다.

5b.6 카메라 간 외부 파라미터가 필요 없습니다

우리 관측은 손바닥 위치 − 큐브 위치 인 **상대량**입니다. 손에도 마커가 있으면 각 카메라가 **독립적으로** 손 기준 큐브 자세를 계산합니다. 그 값은 카메라와 무관하므로, 카메라끼리의 상대 위치를 재거나 맞출 필요가 없습니다.

카메라를 옮기거나 노트북을 건드려도 그대로 동작합니다. 이것이 여러 대를 쓰는 설계에서 가장 큰 이점입니다.

단, 각 카메라가 손 마커를 볼 수 있어야 합니다. 못 보는 각도가 있으면 손등과 손목 양쪽에 마커를 붙이거나 (서로 다른 ID), 그 카메라만 외부 파라미터를 한 번 구해 두십시오.

5b.7 융합

가장 단순하고 튼튼한 방법은 **가장 좋은 하나를 고르는 것**입니다.

1. 검출된 마커 수가 많은 쪽
2. 같으면 재투영 오차가 작은 쪽

평균을 내면 나쁜 추정 하나가 전체를 끌어내립니다. 두 카메라가 모두 2 면 이상을 보고 오차가 비슷할 때만 평균이 이득입니다.

5b.8 끊겼을 때

검출이 0 인 프레임에서는 **직전 값을 유지**하십시오. 정책이 +-2 cm 잡음을 견디므로 몇 프레임 공백은 버팁니다. 끊긴 길이를 로그에 남겨 두십시오 - 그 통계가 5b.9 의 입력이 됩니다.

5b.9 시뮬에 반영할 것

```
random_ori_injection_prob = 0.0    # reorient.py:154
```

확률 p 로 큐브 자세를 **완전히 랜덤한 값**으로 바꿔 관측에 넣는 장치인데 꺼져 있습니다. 마커 오검출, ID 혼동, 가림으로 인한 엉뚱한 해 - 카메라에서 실제로 일어나는 실패가 정확히 이것입니다.

실측한 오검출·끊김 빈도를 넣고 재학습하는 것이 가장 값싼 대비입니다. 지금 정책은 큐브 자세가 항상 그럴듯하다고 가정하고 배웠습니다.

총 루프 지연 (카메라 노출 -> 검출 -> 정책 -> 서보 응답) 도 실측해서 `action_delay_steps` 에 넣으십시오. 9.4 절에 비용이 있습니다.

5b.10 검증

1. 큐브 +z 를 위로 -> `face_up` 이 0 인가
2. 90 도 돌려 +x 를 위로 -> `face_up` 이 2 인가
3. 자로 잰 손-큐브 거리와 추정값이 5 mm 이내인가
4. 큐브를 정지시키고 100 프레임 -> **자세 표준편차가 5.7 도보다 작은가** (학습 시 주입 잡음보다 커지면 정책 입력으로 못 씁니다)
5. 손으로 가려가며 -> 카메라 1 대와 2 대의 **검출 성공률 차이**

5c. 배포 패키지 (Leap_Test)

실기에 필요한 것을 한 곳에 모았습니다. 학습 저장소와 분리되어 있어 실기 PC 로 이 폴더만 복사하면 됩니다.

```
Leap_Test/
  policy/
    policy_np.npz          <- 배포 산출물. numpy 만으로 돈다
    distilled_policy       <- 원본(flax). 재내보내기용
  deploy/
    limits.py              <- 시뮬 env 에서 자동 생성한 안전 범위·상수
    leap_hw.py             <- 드라이버 경계면 (순열 + pi 오프셋 + 서보 I/O)
    obs_builder.py         <- 108차원 관측 조립
```

```
    policy_np.py           <- numpy 추론
    policy_runner.py       <- 20 Hz 제어 루프
    check_joints.py        <- 1단계 개루프 시험 (매핑·부호·지연)
    export_policy.py       <- 증류망 -> npz
  vision/
    README.md              <- 마커 가이드 (인쇄·부착·검증)
    rubik_center/          <- **3x3 루빅큐브. 현재 이것을 쓴다**
      center_markers_A4.pdf   인쇄할 시트(스티커 3종 + 손 마커)
      make_center_templates.py
      cube_board.py           보드 3D 좌표 (공용)
      calibrate_camera.py     카메라 내부 파라미터
      calibrate_face_rot.py   면별 마커 회전
      track_cube.py           자세 추정 + 카메라->월드 변환
      multi_camera.py         여러 대 + 융합
    general_2x2/           <- 일반 큐브용 대안. 지금은 쓰지 않는다
      faces_A4.pdf, markers_A4.pdf, make_*.py
  train/
    run_residual_distill.sh
  tests/
    test_obs_match.py      <- 관측 조립기가 시뮬과 같은 값을 내는지
  docs/
    실기이관_평가_가이드.pdf
```

5c.1 정책이 numpy 로 돕니다

export_policy.py 가 증류망을 가중치와 정규화 통계만 담은 .npz 로 내보냅니다. 실기 PC 에 **jax·brax·orbax 를 설치할 필요가 없습니다.** 필요한 것은 numpy 와 (비전을 쓰면) opencv 뿐입니다.

정규화 통계가 .npz 안에 함께 들어갑니다. 신경망 파라미터만 챙기고 통계를 빠뜨리는 것이 이관에서 가장 흔한 실패인데, 구조적으로 막았습니다.

검증: 무작위 관측 512 개에서 **jax 추론과 numpy 추론의 최대 오차 7.6e-06**.

5c.2 관측 조립기는 시뮬과 대조했습니다

tests/test_obs_match.py 가 시뮬 env 를 굴리면서 같은 입력을 obs_builder 에 넣어 108 차원을 비교합니다. 관측 잡음을 꺼서 결정적으로 만듭니다.

12 스텝 최대 오차 1.7e-07 (float32 정밀도 한계). 순서·부호가 전부 맞습니다. 관측 조립은 실기에서 조용히 틀리기 가장 쉬운 곳이므로, 코드를 고치면 이 시험을 다시 돌리십시오.

5c.3 상수를 손으로 옮기지 않습니다

deploy/limits.py 는 시뮬 env 인스턴스에서 **자동 생성**됩니다. SAFE_LOWERS, SAFE_UPPERS, DEFAULT_POSE, ACTION_SCALE, CTRL_DT, SETTLE_STEPS, 임계값이 들어갑니다. 손으로 적으면 hyperext_floor = -0.2 가 빠지는 식으로 조용히 틀립니다.

5c.4 반드시 재서 넣어야 하는 값 두 개

vision/rubik_center/track_cube.py 상단입니다. 기본값으로 두면 **위치 관측이 그만큼 틀립니다.** 성공 임계값이 15 mm 이므로 무시할 수 없습니다.

	뜻	기본값
PALM_IN_MARKER	손 마커 프레임에서 본 손바닥 (grasp_site) 위치 [m]	0,0,0
HAND_TO_WORLD	손 마커 프레임 -> 시뮬 월드 회전	단위행렬

카메라가 주는 자세는 **카메라 프레임**이고 정책이 먹는 관측은 **월드 프레임** 입니다. 그대로 넣으면 카메라를 어디에 놓느냐에 따라 관측이 통째로 회전합니다. track_cube.py 가 손 마커를 기준으로 돌려 월드로 옮기며, 위 두 값이 그 변환의 나머지 절반입니다.

합성 검증에서 이 변환은 **카메라 위치와 무관하게 같은 값**을 내고, 두 카메라 사이의 불일치가 1e-16 입니다.

5c.4 실행 순서

```
# 0) 설치 (실기 PC)
pip install numpy opencv-contrib-python dynamixel_sdk

# 1) 개루프 - 큐브 없이. 매핑·부호·지연
python deploy/check_joints.py --mode map
python deploy/check_joints.py --mode sign        # 3b.3 의 미확인 3 개
python deploy/check_joints.py --mode latency     # 9.4 표와 대조

# 2) 카메라
python vision/rubik_center/calibrate_camera.py "calib0/*.jpg" camera0.npz
python vision/rubik_center/calibrate_camera.py "calib1/*.jpg" camera1.npz
#    면별 마커 회전 (스크램블 후에도 다시)

# 3) 정지 파지 - 큐브를 올리고 30 초
python deploy/policy_runner.py --cams /dev/v4l/by-id/A:camera0.npz \
                                      /dev/v4l/by-id/B:camera1.npz \
       --marker_mm 12.0 --face_rot 0,3,1,2,0,1 --face 0 --seconds 30

# 4) 연쇄 - 시뮬 지표와 직접 비교되는 숫자가 나온다
python deploy/policy_runner.py --cams ... --chain --seconds 80
```

policy_runner.py 는 매 스텝의 루프 시간을 재서 **예산 (50 ms) 초과 비율**과 **큐브 자세 끊김률**을 끝에 출력합니다. 이 두 숫자가 9.4 절의 지연 표와 5b.10 의 검출 성공률에 각각 대응합니다.

6. 계단식 시험

위험이 낮은 것부터, 앞 단계가 통과해야 다음으로 갑니다.

1 단계—개루프 재생 (큐브 없이)

시뮬에서 기록한 명령 시퀀스를 그대로 실기에 재생하고 엔코더 응답을 겹쳐 봅니다. 정책이 개입하지 않으므로 **순서·부호·영점·지연·추종 특성만** 분리해서 드러납니다.

여기서 안 맞으면 뒤는 볼 필요가 없습니다. 그리고 이 단계가 **작동 지연의 실측값을 주는 단계**이기도 합니다. 시뮬에서 추정치를 쓰는 대신 여기서 잰 값을 넣으십시오.

확인 항목: 관절 순서 대응, 부호 방향, 영점 오프셋, 명령 → 응답 지연 (스텝 수), 정상상태 추종 오차.

2 단계—정지 파지

큐브를 손바닥 위에 올리고 정책을 구동하되 목표를 바꾸지 않습니다. 5 초 유지만 봅니다. 낙하 여부와 손 자세의 안정성만 확인합니다.

3 단계—단일 목표 1 회

한 면만 지정하고 1 회 성공을 목표로 합니다. 성공/실패와 소요 시간을 기록합니다.

4 단계—연쇄

시뮬과 **동일한 지표**로 측정합니다: 연속 달성 횟수, 낙하율, 정체율. 지표가 같아야 시뮬 값과 비교가 성립합니다.

7. 안전

- **토크 상한을 서보 전류 제한으로 강제**하십시오. 소프트웨어 클리핑만으로는 부족합니다.
- **관절 범위를 3 절 값으로 이중 클리핑**하십시오 (정책 출력단과 서보 명령단 양쪽).
- 비상 정지를 손이 닿는 곳에 두십시오.
- 서보 온도를 감시하십시오. 연속 구동에서 과열은 실제로 발생합니다.
- 1 단계 개루프 재생은 **큐브 없이** 하십시오. 규약이 틀린 상태에서 물체를 쥐면 손가락이 서로 충돌할 수 있습니다.

8. 평가 프로토콜

8.1 표본 수—여기서 기대를 낮춰야 합니다

시뮬에서 연속 달성 횟수의 표준편차는 약 2.2 입니다. 실기 N 회의 표준오차는 $2.2/\sqrt{N}$ 입니다.

N	표준오차	95% 신뢰구간
10	0.70	±1.4
30	0.40	±0.8
130	0.19	±0.38

교사 (7.19) 와 residual(6.89) 의 차이 0.30 을 실기에서 가리려면 **약 130 회**가 필요합니다. 사람이 매번 큐브를 올려주는 조건에서 비현실적입니다.

따라서 실기 평가의 목적을"정책 간 비교"로 잡지 마십시오. 정책 선택은 시뮬에서 끝내고, 실기에서는 **"시뮬 성능이 얼마나 남는가"하나만** 봅니다. $N = 30$ 이면 7.19 가 5.5 이하로 떨어지는 것은 검출됩니다. sim2real 격차를 묻는 질문에는 그것으로 충분합니다.

8.2 사전 등록

시행 **전에** 아래를 문서에 적고 시작하십시오. 30 회짜리 표본에서 사후에 기준을 고르면 무엇이든 원하는 결론이 나옵니다.

- 시행 횟수 N 과 중단 조건
- 성공의 정의 (면 오차·위치 임계값, 유지 스텝)
- 낙하·정체의 정의
- 초기 조건 (큐브를 올리는 위치·자세의 허용 범위, 지그 사용 여부)
- 제외 기준 (장비 고장 등, "실패였으니 제외"금지)
- 분석 방법

8.3 시행 절차

- 초기 조건을 지그로 고정하십시오. 사람이 매번 다르게 올리면 시뮬의 리셋 분포 (±1 cm) 와 어긋나고, 그 편차가 결과에 섞입니다.
- 정책이 여럿이면 순서를 무작위화하고, 가능하면 판독자에게 어느 정책인지 가리십시오.
- 서보 온도가 오르면 성능이 변합니다. 시행 간 간격을 일정하게 두십시오.

9. 시뮬 기준선 - 실기에서 무엇과 비교하는가

모든 값은 256 개 환경, 1500 스텝, 연쇄 모드, 목표당 20 스텝 유지 기준입니다. 지표는 **연속 달성 횟수** (리셋 없이 목표를 연달아 몇 번 맞추는가) 입니다.

9.1 정책별 성능 (토크 0.11 N·m)

정책	연속 달성	낙하%	정체%	생존%
교사 팔 F (PPO)	**7.19**	0.8	9.0	90.2
residual SAC (6 개 중 최고)	6.89	1.2	10.2	88.7
증류망 (배포 후보)	5.98	0.8	25.0	74.2

residual SAC 은 **2 시드 x 3 체크포인트 6 개 전부가 교사에 못 미쳤습니다** (6.52~6.89). 학습 곡선도 4 천만 스텝 내내 시작점을 넘지 못했습니다. 낙하율은 교사와 비슷한데 **정체율이 일관되게 높습니다** - 큐브를 떨어뜨리게 만드는 것이 아니라 넘기지 못하고 멈추게 만듭니다.

증류의 목적은 성능 향상이 아니라 **배포 형태 정리**입니다. 9.2 에서 보듯 증류망은 모든 토크에서 교사보다 낮습니다.

9.2 토크 - 지배적인 변수이고, 증류는 손해입니다

토크 (N·m)	교사 팔 F	증류망	차이
0.11	7.19	5.98	-16.8 %
0.16	**12.50**	10.89	-12.9 %
0.21	14.87	13.52	-9.1 %

교사가 모든 토크에서 증류망보다 낮습니다. 증류망의 0.21 에서의 13.52 는 증류의 성과가 아니라 **토크의 성과**였습니다. 교사를 같은 토크에서 재보니 14.87 로 더 높습니다.

그리고 토크 자체의 영향이 압도적입니다. 교사 기준 **0.11 -> 0.16 에서 74%** (7.19 -> 12.50), 0.21 까지 가면 107% 오릅니다.

3b.6 절에서 계산한 실기 동작점이 0.165 N·m 입니다. 즉 우리는 지금까지 **실기보다 훨씬 불리한 조건에서 학습하고 평가해 왔습니다.** 0.11 에서 잰 7.19 는 실기 성능의 하한에 가깝고, 실제로는 12 대가 기대됩니다.

그래서 증류망을 왜 쓰는가

성능이 아닙니다. 세 가지 때문입니다.

1. **단일망** - 지금은 교사 PPO 망과 학생 SAC 망 둘을 동시에 굴려야 합니다.
2. **토크 강건성** - 0.09/0.11/0.16/0.21 에서 증류했으므로 실기 토크를 정확히 모르는 상태에서도 무너지지 않습니다. 교사는 0.11 에서만 학습했습니다.
3. **계획상 SAC 가 주 알고리즘**이라는 요건.

성능만 본다면 **교사 PPO 를 그대로 배포하는 것이 낫습니다.** 배포 대상은 이 셋을 어떻게 저울질하느냐의 문제이고, 9.1 과 이 표가 그 근거입니다.

권고

0.165 N·m 로 다시 학습하십시오. 이것이 지금 가장 값싼 성능 개선입니다. 정책 구조도 보상도 바꾸지 않고 조건만 실기에 맞추는 것이며, 표가 보여주는 개선폭이 다른 어떤 항목보다 큽니다.

9.3 sim2real 항목별 비용

이미 학습된 정책에 **사후로** 걸었을 때의 강건성 비용입니다. 함께 학습하면 통상 이보다 작습니다.

조건	연속 달성	기준 대비	낙하%	정체%
기준 (팔 F)	7.19	-	0.8	9.0
관절 여유 0.1	6.65	-7.5 %	**0.0**	12.5
외력 섭동	6.82	-5.1 %	5.5	9.8

조건	연속 달성	기준 대비	낙하%	정체%
둘 다	6.34	-11.9 %	3.5	12.9

관절 여유 0.1 은 연속 달성을 7.5% 깎는 대신 **낙하를 0.0% 로 없앱니다.** 실기 안전성 관점에서는 유리한 교환일 수 있습니다.

9.4 작동 지연 비용

1 스텝 = 50 ms 입니다. 에피소드마다 `d ~ U{0..K}` 를 뽑아 고정합니다.

K	지연 범위	연속 달성	기준 대비	낙하%	정체%
0	없음	7.43	-	0.0	9.0
1	0~50 ms	7.21	-3.0 %	0.8	9.4
2	0~100 ms	6.85	-7.8 %	1.2	9.8
3	0~150 ms	**5.49**	**-26.1 %**	3.5	**23.4**

K=2 까지는 완만하고 K=3 에서 무너집니다. 정체율이 9.8% 에서 23.4% 로 뛰는 것이 붕괴의 형태입니다.

실기 예산을 대보면:

항목	전형값
FTDI latency timer (기본)	16 ms
카메라 프레임 + 검출	50~100 ms
서보 응답	수 ms

카메라만으로 이미 K=1~2 입니다. 여기에 USB latency 16 ms 를 방치하면 K=3 영역에 들어가 정책이 무너집니다. 10 절의 latency timer 항목이 성능에 직접 연결되는 이유입니다.

10. 이관 전에 메울 구멍

10.1 소프트웨어 / 설정

항목	현재	조치
관절 순서·오프셋	어긋남	3b 절 순열을 **드라이버 경계면**에
`set_ones()` 사용	위험	`q_cmd` 누적 후 `set_leap()`
FTDI latency timer	**기본 16 ms**	**1 ms 로** (9.4 참조)
전류 상한	450 mA = 0.165 N·m	시뮬 0.11 과 불일치, 재학습 판단

```
# latency timer 확인 및 변경
cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
echo 1 ¦ sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
# 영구 적용은 udev 규칙으로
```

10.2 시뮬에 없는 것

항목	현재	실기에서 대응하는 것
옆흔들림 관절 게인	16 개 동일 `kp=3.0`	실기는 ID 8·12·16 만 `kP x 0.75`
관절 영점 오프셋	없음	캘리브레이션 오차

항목	현재	실기에서 대응하는 것
제어 주기 지터	없음	USB 루프가 정확히 20 Hz 로 안 돎
자세 추정 실패	`random_ori_injection_prob = 0`	마커 오검출·가림 (5b.9)
큐브 질량·크기	90 g / 57 mm 고정	**실측해서 교정** (무작위화 아님)

초기 자세에 +-0.05 rad 지터가 있지만 그것은 시작 자세이지 **엔코더 영점 오차가 아닙니다.** 영점 오차는 에피소드 내내 일정하게 유지되는 편향이므로 따로 모형화해야 합니다.

큐브는 무작위화가 아니라 값 교정입니다. 큐브 하나를 고정해 쓴다면 질량은 저울로, 크기는 캘리퍼로 재서 넣으면 됩니다. 무작위화가 정당한 것은 재기 어렵거나 시간에 따라 변하는 항목뿐입니다 - 접촉 마찰 (지문 마모·먼지·유분으로 세션 중에도 변함) 과 무게중심 편심이며, 둘 다 이미 무작위화되어 있습니다.

10.3 실기에서 확인해야 확정되는 것

항목	확인 방법
`if_rot`·`mf_rot`·`rf_rot` 부호	`test_each_joint.py`, 배열 위치 15 (3b.3)
엄지 4 관절 매핑	개루프 재생에서 엄지부터
전원·전류 병목	`test_sequential_load.py` (엄지가 밀리는지)
총 루프 지연	개루프 재생 1 단계
마커 검출 성공률	5b.10

11. 체크리스트

이관 전 - 소프트웨어

- ☐ 정책 파일과 **정규화 통계**를 함께 옮겼다
- ☐ 3b 절 순열 `SIM2ARR` 을 드라이버 경계면에 넣었다 (학습 코드는 그대로)
- ☐ `+3.14159` 오프셋을 적용했다
- ☐ `set_ones()` 를 쓰지 않고 `q_cmd` 를 직접 누적한다
- ☐ pip·dip 하한을 -0.200 으로 클리핑한다
- ☐ 누적을 **명령각**에 한다 (측정각 아님)
- ☐ 추가 필터를 넣지 않았다
- ☐ 제어 주기가 20 Hz 다
- ☐ **FTDI latency timer 를 1 ms 로 바꿨다**
- ☐ 토크 상한을 서보 전류로 강제했다

이관 전 - 비전

- ☐ 마커를 100% 로 인쇄하고 **50 mm 눈금자를 자로 확인**했다
- ☐ 중앙 스티커 크기를 재고 맞는 줄을 썼다
- ☐ 큐브 축을 정하고 ID 0~5 를 해당 면 중앙에 붙였다
- ☐ 손 기준 마커 (ID 40) 를 붙였다
- ☐ 카메라마다 내부 파라미터를 따로 구했다 (재투영 0.5 px 이하)
- ☐ `calibrate_face_rot.py` 로 면별 회전을 구했다
- ☐ 장치 경로를 `/dev/v4l/by-id/` 로 고정했다
- ☐ MJPG 로 강제하고 실제 열린 포맷을 확인했다
- ☐ 노출·초점을 수동 고정했다
- ☐ 최신 프레임만 쓰는 그래버 스레드를 넣었다

시험

- ☐ 1 단계 개루프 재생 통과 (큐브 없이), **총 지연을 실측해 기록**
- ☐ 외전 관절 3 개의 부호를 확인했다
- ☐ 2 단계 정지 파지 통과
- ☐ 3 단계 단일 목표 통과

□ 4 단계 연쇄 - 시뮬과 같은 지표로 측정
□ 마커 검출 성공률을 카메라 1 대/2 대로 비교했다

평가

□ 사전 등록 문서를 작성하고 시작했다
□ 초기 조건을 지그로 고정했다
□ 정책 간 비교가 아니라 sim2real 격차를 목적으로 삼았다